

Computer Programming 143 – Lecture 11

Functions III

Faculty of Engineering



Copyright

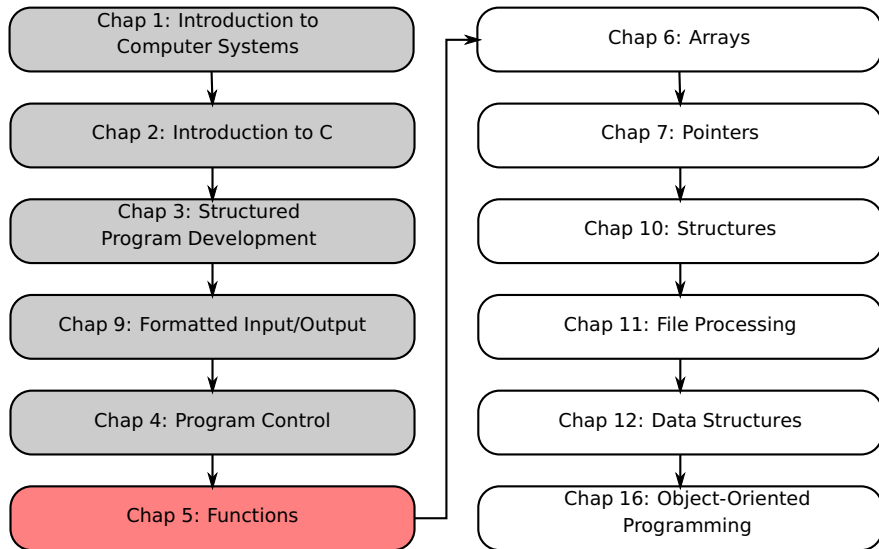
Copyright © 2026 Stellenbosch University
All rights reserved

Disclaimer

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

Module Overview



1 Example: A Game of Chance (5.11)

2 Storage Classes (5.12)

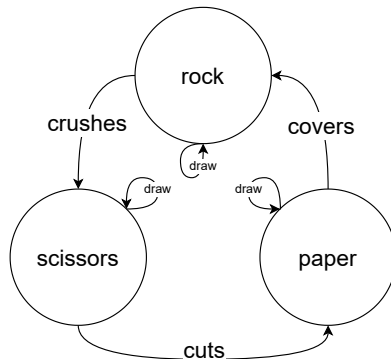
3 Scope Rules (5.13)

5.11 Example: A Game of Chance I

Rules of rock, paper, scissors

Each player simultaneously forms one of three shapes with an outstretched hand. The shapes are "rock", "paper", and "scissors". A simultaneous, zero-sum game, it has three possible outcomes: a draw, a win, or a loss. "rock" breaks (wins) "scissors", "scissors" cuts (wins) "paper", "paper" covers (wins) "rock". If both players choose the same shape, the game is a tie.

5.11 Example: A Game of Chance II



C code

Refer to Fig. 5.7 in Deitel & Deitel

5.11 Example: A Game of Chance III

C code

```
// fig05_07.c
// Simulating the game of rock, paper, scissors
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h> // contains prototype for function time

enum GameStatus {CONTINUE, GAME_WON, GAME_LOST}; // constants represent game
// status
enum RoundStatus {DRAW, WON, LOST}; // constants represent round status
enum Shape {ROCK, PAPER, SCISSORS}; // constants representing possible shapes
enum Shape computerPlayRandomShape(void); // playRandomShape function
// prototype;
enum RoundStatus getRoundStatus(enum Shape, enum Shape);
enum Shape convertIntToShape(int);
char* getStringFromShape(enum Shape);
```

5.11 Example: A Game of Chance IV

C code main function

```
int main(void) {
    srand(time(NULL)); // randomize based on current time
    enum GameStatus gameStatus = CONTINUE; // may be CONTINUE, GAME_WON, or
    // GAME_LOST
    int bestOfN = 3; // best of 3 game
    int winThreshold = (bestOfN / 2) + 1;
    int playerWinCount = 0;
    int computerWinCount = 0;
    int roundNum = 0;

    // while game not complete
    while (CONTINUE == gameStatus) { // should keep playing
        printf("=====\nRound %d\n=====\n", roundNum++);
        printf("Enter 0 (ROCK), or 1 (PAPER), or 2 (SCISSORS): ");

        int playerShapeInt = -1;
        scanf(" %d", &playerShapeInt);
```

5.11 Example: A Game of Chance V

C code main function (cont...)

```
enum Shape playerShape = convertIntToShape(playerShapeInt); // player shape
enum Shape computerShape = computerPlayRandomShape(); // computer plays random shape
enum RoundStatus roundStatus = getRoundStatus(playerShape, computerShape); // get winner
switch (roundStatus) {
    case DRAW:
        printf("It's a DRAW!\n");
        break;
    case WON:
        printf("Player WON!\n");
        if (++playerWinCount == winThreshold) {
            gameStatus = GAME_WON;
        }
        break;
    case LOST:
        printf("Player LOST!\n");
        if (++computerWinCount == winThreshold) {
            gameStatus = GAME_LOST;
        }
        break;
}
```

5.11 Example: A Game of Chance VI

C code main function (cont...)

```
switch (gameStatus) {
    case GAME_WON:
        printf("**Best of %d game is WON by player!**\n", bestOfN);
        break;
    case GAME_LOST:
        printf("**Best of %d game is WON by computer!**\n", bestOfN);
        break;
    case CONTINUE:
    default:
        printf("**Error! Should not reach here!**");
}
return 0;
}
```

5.11 Example: A Game of Chance VII

C code functions (cont...)

```
// play a random shape and display result
enum Shape computerPlayRandomShape(void) {
    enum Shape randomShape = convertIntToShape(rand() % 3);
    printf("Computer played %s\n", getStringFromShape(randomShape));
    return randomShape;
}

enum RoundStatus getRoundStatus(enum Shape playerShape, enum Shape
computerShape) {
    if (playerShape == computerShape) {
        return DRAW;
    } else if ((playerShape == ROCK && computerShape == SCISSORS)
        || (playerShape == SCISSORS && computerShape == PAPER)
        || (playerShape == PAPER && computerShape == ROCK)) {
        return WON;
    } else {
        return LOST;
    }
}
```

5.11 Example: A Game of Chance VIII

C code functions (cont...)

```
enum Shape convertIntToShape(int shapeValue) {
    return (enum Shape) shapeValue;
}

char * getStringFromShape(enum Shape shape) {
    switch (shape) {
        case ROCK:
            return "ROCK";
        case PAPER:
            return "PAPER";
        case SCISSORS:
            return "SCISSORS";
    }
    return "";
}
```

5.11 Example: A Game of Chance IX

Sample output 1

=====

Round 0

=====

Enter 0 (ROCK), or 1 (PAPER), or 2 (SCISSORS): 0

Computer played SCISSORS

player WON!

=====

Round 1

=====

Enter 0 (ROCK), or 1 (PAPER), or 2 (SCISSORS): 1

Computer played PAPER

It's a DRAW!

=====

Round 2

=====

Enter 0 (ROCK), or 1 (PAPER), or 2 (SCISSORS): 2

Computer played ROCK

player LOST!

5.11 Example: A Game of Chance X

Sample output 2

```
=====
Round 0
=====
Enter 0 (ROCK), or 1 (PAPER), or 2 (SCISSORS): 0
Computer played SCISSORS
player WON!
=====
Round 1
=====
Enter 0 (ROCK), or 1 (PAPER), or 2 (SCISSORS): 1
Computer played ROCK
player WON!
**Best of 3 game is WON by player!**
```

Enumerations

```
enum GameState { CONTINUE, GAME_WON, GAME_LOST };
```

- Creates a user-defined enumeration data type
- Specifies a set of integer constants represented by identifiers

```
enum GameState gameState = CONTINUE;
```

- Declares variable **gameStatus** of enumeration type **GameState** and value **CONTINUE**
- Variable **gameStatus** may also take on values **GAME_WON**, or **GAME_LOST**

```
gameStatus = GAME_WON;
```

- Variable **gameStatus** is assigned the value **GAME_WON**

```
if ( gameState == GAME_WON ) {...}
```

- Tests if variable **gameStatus** has the value **GAME_WON**
- Enumerations make code more readable and easily understood

Storage class specifiers

An identifier's storage class determines:

- **Storage duration** – how long a variable exists in memory
- **Scope** – where object can be referenced in program
- **Linkage** – specifies the files in which an identifier is known (more in Chapter 14)

Local variables

- Variables exist for certain section of program (block of code)
- Exist while block is active
- Destroyed when block is exited

Static storage

- Variables exist for entire program execution
- **static**: local variables defined in functions.
 - Keep value after function ends
 - Only known in their own function
- Global variables and functions
 - Known in any function

Example

```
void accumulate(void)
{
    static int acc = 0; // static variable definition
    int num; // local variable definition
    printf("Enter a number: ");
    scanf("%d", &num);
    acc += num; // add number to current total
    printf("The current total is %d\n", acc);
}
```

5.13 Scope Rules I

Definition of scope

Portion of the program in which the identifier can be referenced

File scope

- Variable defined outside a function
- Known in all functions (in file)
- Used for global variables, function definitions, function prototypes

Block scope

- Variable declared inside a block (`{ . . . }`)
- Used for variables, function parameters (local variables of function)
- Outer blocks “hidden” from inner blocks if there is a variable with the same name in the inner block

5.13 Scope Rules II

Example

```
/* Example indicating scope of variable */  
#include <stdio.h>  
  
void useLocal(void); // function prototype  
void useStaticLocal(void); // function prototype  
void useGlobal(void); // function prototype  
  
int x = 1; // global variable
```

Example (cont...)

```
int main(void)
{
    int x = 5; // local variable to main

    printf("local x in scope of main is %d\n", x);
    useLocal(); // useLocal has local x
    useStaticLocal(); // useStaticLocal has static local
    useGlobal(); // useGlobal uses global x
    useLocal(); // useLocal has local x
    useStaticLocal(); // useStaticLocal has static local
    useGlobal(); // useGlobal uses global x
    printf("local x in scope of main is %d\n", x);

    return 0;
}
```

Example (cont...)

```
// useLocal reinitializes local variable x during each call
void useLocal(void)
{
    int x = 25; // initialized each time useLocal is called
    printf("\nlocal x in useLocal is %d ", x);
    printf("after entering useLocal\n");
    ++x;
    printf("local x in useLocal is %d ", x);
    printf("before exiting useLocal\n");
}
```

Example (cont...)

```
/*  
useStaticLocal initializes static local variable x only  
the first time the function is called;  
value of x is saved between calls to this function  
*/  
void useStaticLocal(void)  
{  
    static int x = 50; // initialized only once  
    printf("\nlocal x in useStaticLocal is %d ", x);  
    printf("after entering useStaticLocal\n");  
    ++x;  
    printf("local x in useStaticLocal is %d ", x);  
    printf("before exiting useStaticLocal\n");  
}
```

5.13 Scope Rules VI

Example (cont...)

```
// function useGlobal modifies global variable x during each call
void useGlobal(void)
{
    printf("\nglobal x is %d after entering useGlobal\n", x);
    x *= 10;
    printf("global x is %d before exiting useGlobal\n", x);
}
```

Use globals sparingly! Preferably never

Output:

```
local x in scope of main is 5
```

```
local x in useLocal is 25 after entering useLocal
```

```
local x in useLocal is 26 before exiting useLocal
```

```
local x in useStaticLocal is 50 after entering useStaticLocal
```

```
local x in useStaticLocal is 51 before exiting useStaticLocal
```

```
global x is 1 after entering useGlobal
```

```
global x is 10 before exiting useGlobal
```

5.13 Scope Rules VIII

Output: (cont...)

```
local x in useLocal is 25 after entering useLocal  
local x in useLocal is 26 before exiting useLocal
```

```
local x in useStaticLocal is 51 after entering useStaticLocal  
local x in useStaticLocal is 52 before exiting useStaticLocal
```

```
global x is 10 after entering useGlobal  
global x is 100 before exiting useGlobal
```

```
local x in scope of main is 5
```

Today

Functions III

- Example: a game of chance
- Storage class
- Scope rules

Next lecture

Functions IV

- Recursion

- 1 Study Sections 5.11-5.13 in Deitel & Deitel
- 2 Do Self Review Exercise 5.6 in Deitel & Deitel
- 3 Do Exercise 5.27 in Deitel & Deitel